App.java M | Node.java M | DoubleLinkedList.java M

Node.java > ...

```java
public class Node {
    int data;
    Node next;
    Node prev;

    public Node(int data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}
```

App.java M | Node.java M | DoubleLinkedList.java M

App.java > Language Support for Java(TM) by Red Hat > App > main(String[])

```java
public class App {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        DoubleLinkedList list = new DoubleLinkedList();

        System.out.println(x:"Lista antes de agregar elementos:");
        list.printForward();

        list.add(data:1);
        list.add(data:2);
        list.add(data:5);
        list.add(data:6);

        System.out.println(x:"Lista:");
        list.printForward();

        System.out.println(x:"\n------Imprimir backward------");
        list.printBackward();

        System.out.println(x:"\n------Eliminar elemento 5------");
        list.remove(data:5);
        list.printForward();

        System.out.println(x:"\n------Eliminar elemento 1 (head)------");
        list.remove(data:1);
        list.printForward();

        System.out.println(x:"\n------Eliminar elemento 6 (tail)------");
        list.remove(data:6);
        list.printForward();
    }
}
```

App.java M | Node.java M | DoubleLinkedList.java M

DoubleLinkedList.java > Language Support for Java(TM) by Red Hat > DoubleLinkedList > remove(int)

```java
public class DoubleLinkedList {
    public void printBackward() {
        Node current = tail;
        if (current == null) {
            System.out.println(x:"(Lista vacía)");
            return;
        }
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.prev;
        }
        System.out.println(x:"null");
    }

    public void remove(int data) {
        if (head == null) {
            System.out.println(x:"La lista está vacía, no se puede eliminar.");
            return;
        }

        Node current = head;
        while (current != null && current.data != data) {
            current = current.next;
        }

        if (current == null) {
            System.out.println("Valor " + data + " no encontrado.");
            return;
        }

        if (current == head) {
            head = head.next;
            if (head != null) head.prev = null;
            else tail = null;
            return;
        }

        if (current == tail) {
            tail = tail.prev;
            tail.next = null;
            return;
        }

        current.prev.next = current.next;
        current.next.prev = current.prev;
    }
}
```